

# 1. Introduzione

Benvenuti a tutti in questa presentazione verrà illustrato l'implementazione di un Covert Channel che tramite il protocollo ICMP permetterà la trasmissione di dati fra due macchine. Inizialmente introdurremo il protocollo ICMP e cosa sono i Covert Channel. Successivamente si illustrerà l'implementazione effettuata. Infine verranno illustrati i risultati relativi ai test eseguiti finendo con alcune osservazioni.

## 1. Introduzione ICMP

ICMP è un protocollo che opera al livello di rete e risulta fondamentale per la diagnostica delle reti. Per questo, non può essere completamente disabilitato.

## 2. Introduzione Covert Channel

Un Covert Channel è una comunicazione che permette la comunicazione od il trasferimento di dati in maniera non autorizzata. Solitamente sfruttano vulnerabilità o comportamenti non previsti nei sistemi per non generare segnali di un uso improprio del sistema.

Un Covert Channel viene determinato da alcune caratteristiche.

- **Furtività:** La capacità di evitare di attirare attenzioni su di se.
- **Capacità di trasmissione:** La quantità di dati che il canale trasmette.
- **Indistinguibilità:** Durante la trasmissione il funzionamento delle risorse utilizzate rimane inalterato.

I Covert Channel si dividono in:

- **Timing Covert Channel:** le informazioni vengono inviate attraverso l'istante di tempo in cui avvengono determinati eventi.
- **Storage Covert Channel:** l'informazione viene inserita in un'area di memoria condivisa ed accessibile.
- **Behavioural Covert Channel:** le informazioni vengono codificate mediante gli eventi che avvengono nel sistema.

## 2. Struttura Comunicazione

Ora passiamo ad illustrare lo sviluppo della comunicazione fra le entità.

Per una prima implementazione si è analizzato il progetto **icmp tunnel** per degli spunti. Quindi sarà presente un'entità che farà da proxy fra l'attaccante e la vittima.

### 1. Inizializzazione Entità

Nella fase di inizializzazione:

l'**attaccante** ricaverà

- l'indirizzo IP della vittima
- la lista dei proxy intermedi che si userà per comunicare con la vittima
- il Covert Channel scelto

il **proxy** invece ricava:

- l'indirizzo IP dell'attaccante. Servirà per stabilire la connessione con esso.

la **vittima** infine ricava:

- la quantità di connessioni attive necessarie per l'esfiltrazione dei dati.

Le entità procederanno poi a stabilire le connessioni fra di loro.

## 2. Impostazione della comunicazione fra le entità

L'**attaccante** inizia ricavando con quali proxy può comunicare e che a loro volta riescono a comunicare con la vittima. Quindi:

- si connette ad un proxy ed aspetta che la connessione venga confermata
- Successivamente aspetta l'aggiornamento da proxy; che indicherà se quest'ultimo comunica con la vittima.
- dal risultato l'attaccante decide poi se scartare il proxy.

La procedura termina una volta che si è determinato quali proxy sono utilizzabili. Fra tutti quelli specificati inizialmente.

Il **proxy** invece :

- inizialmente rimane in attesa che l'attaccante si connetta e che gli indichi l'indirizzo della vittima ed il Covert Channel utilizzato.
- successivamente tenterà di creare una comunicazione con la vittima. Aggiungerà poi l'attaccante sul risultato

La **vittima** rimarrà in attesa finché non raggiunge il numero minimo di connessioni stabilite. Durante la connessione, l'entità che si vuole connettere indicherà il Covert Channel che vorrà utilizzare per l'invio dei dati.

## 3. Comunicazione fra le entità

Una volta che tutte le entità sono connesse fra di loro; l'attaccante inserisce un'informazione da mandare alla vittima. Quindi:

- inserisce nel terminale un comando od un messaggio
- sceglie uno dei proxy disponibile che inoltrerà l'informazione (ai proxy restanti indicherà di aspettare i dati dalla vittima)
- successivamente aspetterà che i proxy finiscano di inoltrargli le informazioni ricevute dalla vittima.

Il proxy così come la vittima rimangono in attesa di un comando o di un messaggio.

Il primo quando lo riceve lo inoltra alla vittima e poi aspetta i dati dalla vittima; che verranno poi inoltrati all'attaccante.

La vittima invece:

- procede solamente a confermare la ricezione, se il dato ricevuto è un semplice messaggio.
- se invece si tratta di un comando, procede ad eseguirlo e poi invierà all'attaccante i dati, sfruttando tutte le connessioni disponibili.

## 4. Chiusura della comunicazione

Una volta ricevuti tutti i dati dai proxy, l'attaccante li riordinerà per ricavare il messaggio correttamente. Successivamente deciderà se inviare un ulteriore comando o se terminare la comunicazione.

## 3. TIMING COVERT CHANNEL

Per poter temporizzare i dati da inviare, c'è bisogno di un metodo:

- che calcolerà l'intervallo temporale associato al dato da inviare. Dato quindi un dato restituisce il tempo da aspettare prima di inviare un pacchetto.
- che calcolerà il dato associato ad un intervallo temporale. Dati quindi due pacchetti, calcola l'intervallo temporale e restituisce il dato associato.

### **IMPLEMENTAZIONE DELLA FUNZIONE**

In una prima codifica si ricava il delay basato sulla distanza che i dati devono avere fra di loro.

Si avrà quindi un tempo base, assegnato al primo dato; mentre i successivi avranno una distanza fissa dal dato precedente. Un vantaggio è una maggiore tolleranza ai ritardi di rete; tuttavia lo svantaggio relativo ai tempi che si aspettano rende la codifica sconsigliata.

Per risolvere il problema, inizialmente si sono ridotti i valori inviabili. In particolare si inviano solo i caratteri stampabili (che nella codifica ASCII vanno da 32 a 126). Tuttavia questo ci lega ad una codifica specifica e inoltre porta alla perdita di tanti se mai si mandassero dei dati non relativi a informazioni testuali.

Prendendo spunto da ICMP Exfill si sviluppa meglio l'idea. Il tempo di delay ora viene indicato dal valore intero relativo al byte che si vuole inviare. Dopodiché si normalizzerà il risultato fra un valore minimo ed uno massimo. Tramite questo approccio, il range dei delay viene ristretto in un intervallo preciso e definito. Inoltre permetterà di regolare la velocità di trasmissione tuttavia sacrificando la sensibilità della comunicazione ai ritardi che possono avvenire durante il transito dei pacchetti.

Tuttavia una funzione in cui lo schema dei tempi di invio sembri casuale, permetterà di essere rilevati con maggiore difficoltà. All'implementazione precedente, si aggiungerà quindi del rumore. Segue che il tempo di delay associato a un byte non sarà più costante ma fra un range di valori, nel quale varierà. Ciò però porta che, per potersi scambiare i dati, il mittente e il destinatario devono potersi sincronizzare sulla quantità di rumore che si è aggiunto.

- Il mittente crea un numero randomico indicante il rumore. Quindi le entità si sincronizzeranno sul seed usato oltre al numero di estrazioni effettuate. Se i valori estratti non combaciassero; le entità non riusciranno a scambiarsi i dati correttamente.
- Il mittente indica il rumore generato nel pacchetto stesso. Il ricevente otterrà il valore del rumore generato dalla risorsa condivisa (il pacchetto).

## 4. STORAGE COVERT CHANNEL

In esso la risorsa condivisa sarà il pacchetto ICMP inviato. Il mittente inserirà i dati nei campi del messaggio. Nelle Tabelle sono indicate i campi utilizzati nei messaggi oltre al numero di byte inseribili. Vedremo l'implementazione di alcuni campi, in particolari quelli in cui il dato non può essere inserito direttamente.

### 1. CAMPO HEADER+64 BIT

Nel campo sono presenti l'header IP più 8 byte per una lunghezza complessiva di 28 byte. Nel nostro caso rappresenterà un pacchetto ICMP/IP.

Nell'intestazione IPv4 i dati saranno inseriti in:

- Time to live
- Total length
- Identification

Nell'intestazione ICMPv4 i dati saranno inseriti in:

- Identifier
- Sequence

### 2. CAMPO UNUSED

Dalle specifiche RFC (792 e 4434) il campo deve essere vuoto. Per questo si è utilizzata la libreria struct. Con essa è possibile costruire un pacchetto avente i dati all'interno del campo e successivamente si userà Scapy solo per l'invio.

### 3. CAMPO TIMESTAMP

Contiene i millisecondi dalla mezzanotte UT nel campo il byte viene inserito nella parte dei millisecondi. Non è possibile inserire ulteriori informazioni nelle altre parti del campo siccome rappresentano le ore, i minuti ed i secondi. L'alternativa di inserire il dato in tutto il campo porterebbe a tempi errati.

### 4. CAMPO INVOKING PACKET

Come per Header+64 bit, indica il pacchetto che ha causato l'errore. . Nel nostro caso si userà l'intestazione IPv4 che è di 40 byte, ed il protocollo ICMPv4, la cui intestazione è di 8 byte.

Nell'intestazione IPv6 i dati saranno inseriti in:

- Payload Length
- Next Header
- Hop Limit

Nell'intestazione ICMPv6 i dati saranno inseriti in:

- Identifier
- Sequence

## 5. BEHAVIOURAL COVERT CHANNEL

Nell'implementazione, l'evento sarà definito dall'arrivo di un messaggio ICMP. Nella Tabella vengono indicate le coppie (Tipologia,Codice) ed il relativo dato associato. Siccome il numero di eventi possibili sono 17, è possibile codificare solo 4 bit alla volta. L'evento che rimane inutilizzato, verrà usato per indicare l'inizio e la terminazione dell'invio dei dati.

Un problema di questa possibile implementazione, sono le tipologie deprecate dei messaggi. Questi messaggi potranno comunque essere inviati ma le linee guida indicano ai router di ignorarli e all'host utente di non mandarli. Inoltre, date le linee guida, non è assicurata la ricezione dei messaggi.

- Information Request/Reply è stato deprecato siccome meccanismi come il DHCP, lo hanno sostituito per la configurazione dell'host.
- Source Quench: era concepito come meccanismo per il controllo della congestione. Tuttavia, data la sua inefficienza è stato formalmente deprecato dal 1995. è stato sostituito dai meccanismi implementati nel protocollo TCP.

## TEST E RISULTATI

L'invio di un messaggio di tipo Request richiede la ricezione di una risposta; che normalmente sarà uguale al messaggio inviato. Una soluzione può essere l'invio di soli messaggi di tipo Reply oppure la disabilitazione delle risposte così da poterne inviare una noi stessi.

Inizialmente si sono effettuati test in cui si inviava il dato una singola volta. Dai risultati ottenuti si ottiene che RITA non hanno mostrato la presenza dei Covert Channel.

Uno step successivo è l'invio degli stessi dati più volte (in particolare tre volte). Inoltre si sono definiti dei parametri per verificare meglio cosa avviene:

- La quantità di dati inviati (1KB, 10KB, 100KB, 1MB).
- I campi usati dal Covert Channel. Per esempio nei messaggi Icmp Echo le variazioni si baseranno sull'utilizzo o meno del campo Identifier o del campo Data.
- Il delay fra i blocchi di dati. Siccome i dati verranno suddivisi in blocchi da 100KB; si può decidere se avere un invio sequenziale dei blocchi oppure se avere un delay fra un blocco ed

il successivo. La divisione in blocchi permette l'invio di file con grosse dimensioni evitando di congestionare il canale di comunicazione.

- Tempo di riposo prima di un nuovo invio (per esempio 1 ora, 30 minuti, 15 minuti).
- Il mittente del messaggio. Nel messaggio si può inserire il mittente reale oppure si potranno indicare mittenti diversi; così da far sembrare che il pacco sia stato spedito da un altro host. Nel caso gli indirizzi verranno presi da un pool indicando o gli indirizzi IP attualmente assegnati a degli host nella rete oppure gli indirizzi IP non associati ad alcun dispositivo.

Dai risultati si nota che la modifica del mittente e tempi di attesa lunghi prima di un successivo invio premettono di ridurre la gravità assegnata alla connessione e in alcuni casi anche il Beacon score. I test in cui si utilizzava solamente l'indirizzo IP assegnato all'host, hanno avuto risultati peggiori rispetto ai test in cui si sono utilizzati più mittenti. Invece tempi di attesa abbastanza lunghi fra un invio ed un altro hanno ridotto la gravità ed il beacon score.

Un'altra osservazione è relativa sull'invio sequenziale dei blocchi dei dati. Nei test non sono mai stati identificati da RITA. Tuttavia una grande mole di dati congestionerebbe il canale di comunicazione, con la conseguente creazione di rumore; che porterà l'attenzione, da parte degli amministratori e dei sistemi di sicurezza, sul Covert Channel. Invece per dimensioni minori potrebbe essere una strategia utilizzabile. Infatti nei test che inviavano dati sugli ordini dei KiloByte, sia un invio sequenziale che ritardato dei blocchi non veniva rilevato.

## **CONCLUSIONI SULL'IMPLEMENTAZIONE DEI COVERT CHANNEL**

In conclusione, in questa presentazione, si è visto come il protocollo ICMP, fondamentale per la diagnostica delle reti, può essere utilizzato per la creazione di un canale di comunicazione fra più entità; permettendo, inoltre, l'implementazione di tre tipologie di Covert Channel.

Dai test effettuati, invece, si è ricavato che la rivelabilità del canale dipendeva:

- dalla frequenza di trasmissione dei dati.
- dalla quantità di pacchetti mandati da una sorgente in un intervallo di tempo.

Modificando quindi il mittente del pacchetto e impostando un giusto tempo di attesa (prima di un successivo invio) si è riusciti a diminuire la gravità del Covert Channel.

Quindi date le sue funzionalità il protocollo non può essere completamente filtrato. Tuttavia si possono effettuare delle accortezze:

- permettere, nel traffico di rete, solo le tipologie di messaggi ICMP essenziali.

Fra queste bisognerà monitorare il loro utilizzo e il contenuto:

- si dovranno prevedere metodi per l'ispezione dei pacchetti.

- bisogna limitare la quantità di messaggi che un host può inviare in un determinato intervallo di tempo
- aggiungere del ritardo ai messaggi entranti prima di inoltrarli.

Infine per vedere se il mittente sta inviando i dati secondo uno schema anomalo si possono effettuare delle analisi statistiche rispetto al flusso di rete nelle condizioni normali.